

Blocco 9 - CTE, viste e manutenibilità

Rendere leggibili query che crescono

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Rappresentazione iniziale

Partiamo da una soluzione semplice e concreta, prima di introdurre il modello generale.

Perché questo blocco

Problema didattico

Le query reali tendono a crescere: join, filtri, aggregazioni, casi particolari. Una query monolitica funziona forse oggi, ma diventa difficile da leggere, correggere e spiegare.

Idea guida

Prima osserviamo una rappresentazione naturale, poi ne discutiamo i limiti e solo dopo formalizziamo.

Una query lunga in un solo blocco

Situazione concreta

Una query reale può contenere filtri, join, calcoli e aggregazioni. Scriverla tutta insieme rende difficile capire dove nasce un errore.

```
SELECT ...  
FROM ... JOIN ...  
WHERE ...  
GROUP BY ...  
ORDER BY ...;
```

Limiti della rappresentazione iniziale

- ▶ poco leggibile;
- ▶ difficile da testare a passaggi;
- ▶ logica duplicata in query diverse;
- ▶ nomi intermedi assenti.

2. Soluzione più generale

Riorganizziamo l'informazione in una forma più flessibile e controllabile.

Da blocco unico a passaggi nominati

Principio

CTE e viste permettono di dare nome ai passaggi logici: filtro, arricchimento, aggregazione e risultato finale.

Esempio di riorganizzazione

```
WITH valid_orders AS (...),  
     order_totals AS (...)  
SELECT ...  
FROM order_totals;
```


Processo di trasformazione

- ① scrivere la domanda finale;
- ② dividere in passaggi logici;
- ③ dare nomi descrittivi alle CTE;
- ④ testare ogni passaggio;
- ⑤ promuovere a vista solo logica stabile.

Analogia: ricetta a fasi

Analogia o caso esplicativo

Una ricetta complessa non dice solo “prepara la cena”: separa base, salsa, cottura e impiattamento. Le CTE fanno lo stesso con una query: danno nome a passaggi intermedi verificabili.

3. Formalizzazione

Diamo un nome preciso ai concetti emersi dall'esempio.

- ▶ CTE come relazione temporanea nominata;
- ▶ vista come query salvata;
- ▶ responsabilità singola;
- ▶ contratto logico;
- ▶ test intermedi.

- ▶ **CTE**: risultato temporaneo nominato dentro una query.
- ▶ **Vista**: query salvata come oggetto logico interrogabile.
- ▶ **Responsabilità singola**: ogni CTE fa una cosa sola.
- ▶ **Contratto**: colonne, granularità e filtri garantiti da una vista.
- ▶ **Test intermedio**: esecuzione di un passaggio per verificarne righe e significato.

Come riconoscere una buona soluzione

- ▶ ogni CTE ha nome, input, output e granularità chiari;
- ▶ la query finale si legge dall'alto verso il basso;
- ▶ la vista espone una semantica stabile;
- ▶ i passaggi intermedi sono testabili singolarmente.

4. Esempio guidato

Applichiamo il modello generale a un caso piccolo, leggendo ogni passaggio.

Esempio: situazione iniziale

Domanda o frase di dominio

Domanda: “Ricavo mensile per paese, calcolato solo sugli ordini validi, con nome cliente e canale disponibili per controlli.”

Esempio: ragionamento

- ▶ `valid_orders`: filtra stati non validi;
- ▶ `line_amounts`: calcola importi di riga;
- ▶ `order_totals`: torna a una riga per ordine;
- ▶ `country_month`: aggrega al livello richiesto.

Esempio completo: CTE a responsabilità singola

```
WITH valid_orders AS (  
    SELECT order_id, customer_id, order_date, channel  
    FROM orders  
    WHERE status NOT IN ('cancelled', 'refunded')  
) , line_amounts AS (  
    SELECT order_id,  
           quantity * unit_price * (1 - discount_pct / 100.0) AS net_amount  
    FROM order_items  
) , order_totals AS (  
    SELECT order_id, round(sum(net_amount), 2) AS revenue  
    FROM line_amounts  
    GROUP BY order_id  
)  
SELECT date_trunc('month', v.order_date)::date AS month,  
       c.country,  
       round(sum(t.revenue), 2) AS revenue  
FROM valid_orders v  
JOIN order_totals t ON t.order_id = v.order_id  
JOIN customers c ON c.customer_id = v.customer_id  
GROUP BY month, c.country  
ORDER BY month, c.country;
```

Errori frequenti da prevenire

- ▶ usare CTE come contenitore di codice disordinato;
- ▶ non controllare la granularità di ogni passaggio;
- ▶ creare viste con nomi generici e semantica poco chiara;
- ▶ duplicare la stessa definizione KPI in file diversi;
- ▶ nascondere filtri business dentro una vista senza documentarli.

5. Pratica guidata

Ripetiamo lo stesso processo su un problema diverso: rappresentazione iniziale, limiti, riorganizzazione, soluzione.

Esercitazione: problema informale

Contesto

Una query KPI è diventata lunga e fragile. Il compito è renderla leggibile e riusabile, senza cambiare il risultato.

Esercitazione: specifica corretta

- ▶ input: query aggregata su ordini, righe e clienti;
- ▶ output: versione con CTE e, quando opportuno, vista di supporto;
- ▶ vincolo: ogni CTE deve avere responsabilità singola;
- ▶ vincolo: la granularità di ogni passaggio va dichiarata in commento o nel nome.

Esercitazione: definizione della soluzione

- ① isolare filtri di validità ordine;
- ② isolare calcolo importi di riga;
- ③ aggregare a ordine prima di aggregare a mese o paese;
- ④ creare una vista solo per una definizione stabile e riusabile.

Implementazione completa: vista riusabile (1/2)

```
SET search_path TO training;

CREATE OR REPLACE VIEW valid_order_revenue AS
SELECT r.order_id, r.customer_id, r.order_date, r.channel,
       r.status, r.gross_revenue
FROM order_revenue r
WHERE r.status NOT IN ('cancelled', 'refunded');

WITH monthly_channel AS (
  SELECT date_trunc('month', order_date)::date AS month,
         channel,
         round(sum(gross_revenue), 2) AS revenue
  FROM valid_order_revenue
  GROUP BY month, channel
)
SELECT month, channel, revenue
FROM monthly_channel
ORDER BY month, channel;
```


Implementazione completa: vista riusable (2/2)

```
WITH customer_revenue AS (  
    SELECT customer_id, round(sum(gross_revenue), 2) AS revenue  
    FROM valid_order_revenue  
    GROUP BY customer_id  
) , enriched AS (  
    SELECT c.customer_id, c.full_name, c.segment,  
           coalesce(cr.revenue, 0) AS revenue  
    FROM customers c  
    LEFT JOIN customer_revenue cr ON cr.customer_id = c.customer_id  
)  
SELECT *  
FROM enriched  
ORDER BY revenue DESC, customer_id;
```

Esercitazione: organizzazione dei 90 minuti

- ▶ 15 min leggere la query monolitica;
- ▶ 25 min identificare passaggi logici;
- ▶ 25 min riscrivere con CTE;
- ▶ 15 min creare o discutere una vista;
- ▶ 10 min confronto su naming e test.

- ▶ confrontare almeno due soluzioni quando esistono alternative;
- ▶ chiedere quali assunzioni cambierebbero la soluzione;
- ▶ separare errori di sintassi, errori di modello ed errori di interpretazione.

- ▶ una CTE utile ha un nome che spiega il dato prodotto;
- ▶ la mantenibilità si misura dalla facilità con cui un altro lettore verifica la query;
- ▶ una vista è un contratto logico: va usata quando la semantica è stabile.